C to Kotlin: Programming Concepts Comparison

By Vikramaditya Khupse

1. Basic Program Structure

C

c Copy

```c
#include <stdio.h>

// Main function is the entry point
int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Kotlin

kotlin Copy

```kotlin
// Main function is more concise
fun main() {
    println("Hello, World!")
}
```

2. Variables and Data Types

C

c Copy

```c
#include <stdio.h>

int main() {
    // Explicit type declaration
    int integerVar = 10;
    float floatVar = 3.14f;
    char charVar = 'A';

    // Pointer declaration
    int* pointerVar = &integerVar;

    printf("Integer: %d\n", integerVar);
    printf("Float: %f\n", floatVar);
    printf("Char: %c\n", charVar);
    printf("Pointer value: %d\n", *pointerVar);

    return 0;
```

Kotlin

kotlin Copy

```kotlin
fun main() {
    // Type inference
    val immutableVar = 10 // Immutable (like const)
    var mutableVar = 20   // Mutable variable

    // Explicit type declaration
    val explicitInt: Int = 30
    val floatVar: Float = 3.14f
    val charVar: Char = 'A'

    // No direct pointer manipulation
    println("Integer: $immutableVar")
    println("Float: $floatVar")
    println("Char: $charVar")
}
```

3. Control Structures - Conditionals

```c
// C

#include <stdio.h>

int main() {
    int x = 10;

    // If-else statement
    if (x > 5) {
        printf("x is greater than 5\n");
    } else if (x == 5) {
        printf("x is equal to 5\n");
    } else {
        printf("x is less than 5\n");
    }

    // Switch statement
    switch (x) {
        case 10:
            printf("x is 10\n");
            break;
        case 5:
            printf("x is 5\n");
            break;
        default:
            printf("x is something else\n");
    }

    return 0;
}
```

```kotlin
//Kotlin
fun main() {
    val x = 10
    // If-else as an expression
    val result = if (x > 5) {
        "x is greater than 5"
    } else if (x == 5) {
        "x is equal to 5"
    } else {
        "x is less than 5"
    }
    println(result)

    // When statement (more powerful than switch)
    when (x) {
        10 -> println("x is 10")
        5 -> println("x is 5")
        else -> println("x is something else")
    }

    // Range-based when
    when (x) {
        in 0 ≤ .. ≤ 9 -> println("x is between 0 and 9")
        in 10 ≤ .. ≤ 20 -> println("x is between 10 and 20")
    }
}
```

4. Loops

c

Copy

```c
#include <stdio.h>

int main() {
    // For loop
    for (int i = 0; i < 5; i++) {
        printf("For loop iteration: %d\n", i);
    }

    // While loop
    int j = 0;
    while (j < 5) {
        printf("While loop iteration: %d\n", j);
        j++;
    }

    // Do-while loop
    int k = 0;
    do {
        printf("Do-while loop iteration: %d\n", k);
        k++;
    } while (k < 5);

    return 0;
```

```kotlin
//Kotlin
fun main() {
    // For loop with range
    for (i in 0 ≤ until < 5) {
        println("For loop iteration: $i")
    }

    // While loop (similar to C)
    var j = 0
    while (j < 5) {
        println("While loop iteration: $j")
        j++
    }

    // Repeat loop (alternative to do-while)
    var k = 0
    repeat(5) {
        println("Repeat loop iteration: $k")
        k++
    }

    // Iterating over collections
    val list = listOf(1, 2, 3, 4, 5)
    for (item in list) {
        println("List item: $item")
    }
}
```

5. Functions

c

Copy

```c
#include <stdio.h>

// Function declaration
int add(int a, int b) {
    return a + b;
}

// Function with multiple parameters
void printDetails(char* name, int age) {
    printf("Name: %s, Age: %d\n", name, age);
}

int main() {
    int sum = add(5, 3);
    printf("Sum: %d\n", sum);

    printDetails("John", 25);

    return 0;
}
```

kotlin

Copy

```kotlin
// Top-level function
fun add(a: Int, b: Int): Int {
    return a + b
}

// Function with default parameters
fun printDetails(name: String, age: Int = 0) {
    println("Name: $name, Age: $age")
}

// Single-expression function
fun multiply(a: Int, b: Int) = a * b

fun main() {
    val sum = add(5, 3)
    println("Sum: $sum")

    printDetails("John", 25)
    printDetails("Jane") // Age defaults to 0

    val product = multiply(4, 6)
    println("Product: $product")
}
```

6. Memory Management

```c
// C
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Dynamic memory allocation
    int* dynamicArray = (int*)malloc(5 * sizeof(int));

    if (dynamicArray == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    // Using the dynamic array
    for (int i = 0; i < 5; i++) {
        dynamicArray[i] = i * 10;
    }

    // Print array
    for (int i = 0; i < 5; i++) {
        printf("Element %d: %d\n", i, dynamicArray[i]);
    }

    // Always free dynamically allocated memory
    free(dynamicArray);

    return 0;
}
```

```kotlin
//Kotlin
fun main() {
    // Lists are dynamically sized
    val list = mutableListOf<Int>()

    // Adding elements
    for (i in 0 ≤ until < 5) {
        list.add(i * 10)
    }

    // Printing list
    list.forEachIndexed { index, value ->
        println("Element $index: $value")
    }

    // No manual memory management needed
    // Kotlin uses garbage collection

    // Nullable types
    var nullableInt: Int? = null
    nullableInt = 42

    // Safe call operator
    println(nullableInt?.toString())
}
```

7. Structs vs Classes

```c
// C
#include <stdio.h>
#include <string.h>

// Struct definition
struct Person {
    char name[50];
    int age;
};

// Function to initialize struct
void initializePerson(struct Person* p, const char* name, int age) {
    strcpy(p->name, name);
    p->age = age;
}

int main() {
    // Creating a struct instance
    struct Person person1;
    initializePerson(&person1, "John Doe", 30);

    // Printing struct details
    printf("Name: %s, Age: %d\n", person1.name, person1.age);

    return 0;
}
```

```kotlin
//Kotlin
// Class definition with constructor
class Person(val name: String, var age: Int) {
    // Method within the class
    fun introduce() {
        println("Name: $name, Age: $age")
    }
}

// Data class (automatic toString, equals, hashCode)
data class Student(val name: String, val grade: Int)

fun main() {
    // Creating class instances
    val person1 = Person("John Doe", 30)
    person1.introduce()

    // Data class usage
    val student1 = Student("Alice", 10)
    println(student1) // Automatic toString

    // Copying with modifications
    val student2 = student1.copy(grade = 11)
    println(student2)
}
```

8. Error Handling

```c
// C
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

int divideNumbers(int a, int b) {
    if (b == 0) {
        // Set errno for error indication
        errno = EINVAL;
        return -1;
    }
    return a / b;
}

int main() {
    int result =

    if (result =
        // Check
        fprintf(stderr, "Error: %s\n", strerror(errno));
        return 1;
    }

    printf("Result: %d\n", result);

    return 0;
}
```

```
#define stderr stderr
Expands to:
stderr
```

```kotlin
//Kotlin
// Exception handling
fun divideNumbers(a: Int, b: Int): Int {
    // Explicit exception throwing
    require(b != 0) { "Divisor cannot be zero" }
    return a / b
}

fun main() {
    // Try-catch block
    try {
        val result = divideNumbers(10, 0)
        println("Result: $result")
    } catch (e: IllegalArgumentException) {
        println("Error: ${e.message}")
    } finally {
        println("Division attempt completed")
    }

    // Nullable and safe calls
    val nullableValue: Int? = null
    println(nullableValue?.toString() ?: "Value is null")
}
```

Key Differences to Remember

Memory Management:

- C requires manual memory management
- Kotlin uses garbage collection

Type System:

- C has static typing with explicit declarations
- Kotlin has type inference and nullable types

Syntax:

- C is more verbose
- Kotlin aims for conciseness and readability

Paradigm:

- C is primarily procedural
- Kotlin supports both object-oriented and functional programming